Combinatorial Algorithms (Math 482), 2026 Spring Problem Set 6

Due by Wednesday, March 4, 2026.

1. **Greedy Knapsack.** Recall that in the standard version of the knapsack problem, we are given n items with values $V[1..n]$, weights $W[1..n]$ and a capacity C; and we need to choose items to maximize the value subject to the capacity constraint. We have seen in class, that a greedy algorithm finds an optimal solution if all items have unit weight.

   (a) Suppose that all items have unit value (but arbitrary weight). Design a greedy algorithm for this problem. Prove that your algorithm finds an optimal solution.

   (b) Suppose that the value of each item is 1 or 3 (weights are arbitrary). Does your greedy algorithm still work? Either prove that it does or find a counterexample.

   (c) Suppose that the value of every item is 2 or 3 (weights are arbitrary). Does your greedy algorithm still work? Either prove that it does or find a counterexample.

2. **Modified Interval Scheduling.** Recall that in the interval scheduling problem, we are given n requests/intervals $(S[1], F[1]), \ldots, (S[n], F[n])$ with values $V[1..n]$, and we need to find a conflict-free subset of maximum value. In class, we discussed a greedy algorithm for the unweighted version of the problem (where all intervals have unit value).

   (a) Does the greedy algorithm still work if the value of each request/interval is 1 or 2? Either prove that it does or find a counterexample.

   (b) Now suppose that the values $V[1..n]$ are arbitrary, but all intervals have the unit length (that is, $F[i] - S[i] = 1$ for all $i = 1, \ldots, n$). Does the greedy algorithm work in this case? Can it be adapted to solve the problem in $O(n \log n)$ time?

3. **Interval Coloring.** Recall that in the interval coloring problem, we are given n requests/intervals $(S[1], F[1]), \ldots, (S[n], F[n])$, and we need to assign each interval to a room ("color"), such that there is no conflict between intervals assigned to the same room, and total number of rooms is minimized. In class, we discussed a greedy algorithm for this problem.

   Design data structures that support this algorithm (e.g., how to find an unoccupied room at a given time?), write pseudo-code for the algorithm, and analyze running time.

4. **Interval Coloring with Small and Large Rooms.** Consider the following version of the Interval Coloring Problem. There are two types of rooms: *Large rooms* (50 seats) and *Small rooms* (25 seats). The requests come with a binary array $B[1..n]$, where $B[i] = 1$ means that request i must be assigned to a large room, and $B[i] = 0$ means that request i can be assigned to any room. We need to assigns the requests to rooms such that all requests for large rooms are honored, and the objective function $R_S + 2\,R_L$ is minimized, where R_S and R_L, respectively, denote the number of small and large rooms that we use.

   Design an efficient algorithm for this problem. Prove that your algorithm finds an optimal solution. What is the running time of your algorithm?